National University
of computer and emerging sciences

MLOps

Assignment #02

Submitted by:

Hijab Tahira Beg 22i-2071

Technical Report: Airflow + MLflow Titanic Survival Pipeline

1. Architecture Explanation (Airflow + MLflow Interaction)

This project implements an end-to-end machine learning pipeline for predicting Titanic passenger survival using Apache Airflow for orchestration and MLflow for experiment tracking and model management. The architecture integrates data processing, model training, evaluation, and model registry into a reproducible workflow.

Airflow acts as the pipeline orchestrator, coordinating the sequence of tasks required to transform the raw dataset into a trained machine learning model. Each stage of the pipeline is implemented as a separate Airflow task using `PythonOperator`.

MLflow is used for experiment tracking and model lifecycle management. During the model training stage, the pipeline logs experiment metadata, hyperparameters, evaluation metrics, and trained models to the MLflow tracking server. This enables comparison across multiple experiments and ensures reproducibility.

The interaction between Airflow and MLflow occurs primarily during the training and evaluation stages. When the training task executes, it connects to the MLflow server using the tracking URI:

```
http://host.docker.internal:5000
```

This allows the Airflow container to communicate with the MLflow service running on the host machine. The training task logs:

- hyperparameters
- dataset characteristics
- trained model artifacts
- evaluation metrics

These records are stored as MLflow runs under the experiment `Titanic_Airflow_MLflow_Assignment`. After model evaluation, the pipeline applies branching logic to determine whether the trained model should be registered in the MLflow Model Registry. If the model meets the performance threshold, it is registered as a new version of the model.

This architecture ensures that workflow orchestration, experiment tracking, and model management are fully integrated, creating a production-style machine learning pipeline.

2. DAG Structure and Dependency Explanation

The pipeline is implemented as a **Directed Acyclic Graph (DAG)**.

The DAG consists of several sequential stages representing the machine learning workflow.

1. Data Ingestion

The pipeline begins with the data_ingestion task, which loads the Titanic dataset from the mounted data directory. This task verifies that the dataset exists and prints basic dataset statistics such as shape and missing values. The dataset path is then passed to downstream tasks using Airflow's XCom mechanism.

2. Data Validation

The data_validation task performs integrity checks on the dataset. Specifically, it calculates the percentage of missing values in the Age and Embarked columns. If the missing value percentage exceeds a predefined threshold, the pipeline raises an exception.

An intentional failure is implemented in the first attempt to demonstrate Airflow's retry mechanism. On retry, the validation proceeds normally.

3. Data Preprocessing

After validation, two preprocessing tasks run in parallel:

Handle Missing Values

- Missing Age values are replaced with the column median.
- Missing Embarked values are replaced with the most frequent value.

Feature Engineering
Two additional features are created:

- FamilySize
- IsAlone

These features capture social relationships among passengers.

4. Data Encoding

The encode_data task merges engineered features with the main dataset and performs categorical encoding. The Sex column is converted into numerical values, and one-hot encoding is applied to the Embarked column.

Irrelevant columns such as name, ticket, and cabin are removed to prepare the dataset for machine learning.

5. Model Training

The train_model task trains a Logistic Regression model using Scikit-learn. Hyperparameters are logged to MLflow along with dataset statistics and the trained model artifact.

The dataset is split into training and testing sets using a stratified split to maintain class balance.

6. Model Evaluation

The evaluate_model task computes several evaluation metrics:

- Accuracy

- Precision
- Recall
- F1 Score

These metrics are logged to MLflow for experiment comparison.

7. Branching Decision

The `branch_on_accuracy` task determines the next pipeline step. If model accuracy is at least **0.80**, the model is registered in MLflow. Otherwise, the model is rejected.

8. Model Registration or Rejection

Two possible outcomes follow:

register_model
Registers the trained model in the MLflow Model Registry.

reject_model
Tags the run as rejected if the model fails to meet the accuracy threshold.

This branching logic ensures that only sufficiently accurate models are promoted to the registry.

3. Experiment Comparison Analysis

Three experiments were conducted using Logistic Regression with different hyperparameters.

Experiment Hyperparameters

Parameter	Experiment 3	Experiment 2	Experiment 1
model_type	LogisticRegression	LogisticRegression	LogisticRegression
max_iter	1000	500	200
C	0.5	default	default
solver	liblinear	default	default
dataset_size	891	891	891
feature_count	11	11	11
test_size	0.2	0.2	0.2
random_state	42	42	42

Evaluation Metrics

Metric	Experiment 3	Experiment 3	Experiment 1
Accuracy	0.827	0.804	0.816
F1 Score	0.756	0.729	0.740
Precision	0.828	0.783	0.810
Recall	0.696	0.681	0.681

Analysis

Experiment 3 achieved the best overall performance. It used a higher number of iterations (max_iter = 1000) along with stronger regularization (C = 0.5) and the liblinear solver. These settings improved the optimization process and allowed the model to converge more effectively.

Experiment 2 showed the lowest performance among the three runs. Reducing the number of iterations to 500 likely limited the model's ability to fully converge, which resulted in slightly worse accuracy and F1 score.

Experiment 1 performed better than Experiment 2 but still slightly worse than Experiment 3. With only 200 iterations, the model may not have been given enough training cycles to reach optimal convergence.

Overall, Experiment 3 produced the most reliable model, achieving the highest accuracy, precision, recall, and F1 score.

Despite the lower performance of Experiment 2, the model was still registered because the pipeline uses an **accuracy threshold of 0.80** as the acceptance criterion. Since Experiment 2 achieved an accuracy of **0.804**, it still satisfied the registration condition.

4. Failure and Retry Explanation

The pipeline intentionally demonstrates Airflow's failure handling and retry mechanism during the data validation stage.

In the data_validation task, the code deliberately triggers a failure on the first execution attempt:

```
if force_retry_demo and ti.try_number == 1:
    raise AirflowException("Intentional validation failure...")
```

This simulates a common real-world scenario where a pipeline may fail due to temporary issues such as data availability or infrastructure instability.

Airflow is configured to retry the validation task once:

```
retries = 1
```

```
retry_delay = timedelta(minutes=1)
```

When the first attempt fails, Airflow marks the task as failed and schedules a retry after the specified delay. On the second attempt, the validation proceeds normally and the pipeline continues.

This mechanism ensures that pipelines remain robust against temporary failures and reduces the need for manual intervention.

5. Reflection on Production Deployment

If this pipeline were deployed in a production environment, several improvements would be necessary.

First, the MLflow tracking server would be deployed as a dedicated service with a persistent backend database such as PostgreSQL. This would ensure reliable experiment storage and model registry management.

Second, model selection logic should be enhanced. The current pipeline registers any model with accuracy above 0.80. In production systems, models are typically compared against previous versions, and only the best performing model is promoted.

Third, automated monitoring should be implemented. Production pipelines often include data drift detection and model performance monitoring to ensure that models remain accurate as new data becomes available.

Additionally, container orchestration tools such as Kubernetes could be used to deploy the Airflow environment at scale. This would allow multiple pipelines to run concurrently while maintaining reliability and fault tolerance.

Finally, automated CI/CD pipelines could be integrated to manage version control, testing, and deployment of machine learning models.